Московский авиационный институт (национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Курсовой проект по курсу «Дискретный анализ»

Студент: Д. А. Широбоков
Преподаватель:
Группа: М8О-301БВ
Дата:
Оценка:
Подпись:

Москва, 2026

I. Персистентные структуры данных

Задача: Дан набор горизонтальных отрезков, и набор точек. Для каждой точки определить сколько отрезков лежит строго над ней.

Решение должно работать online, то есть должно обрабатывать запросы по одному после построения необходимой структуры данных по входным данным. Чтение входных данных и запросов вместе и построение по ним общей структуры запрещено.

Формат ввода

В первой строке вам даны два числа n и m ($1 \le n, m \le 10^5$) — количество отрезков и количество точек соответственно. В следующих n строках вам заданы отрезки, в виде троек чисел l, r и h ($-10^9 \le l < r \le 10^9, -10^9 \le g \le 10^9$) — координаты x левой и правой границ отрезка и координата y отрезка соответственно. В следующих m строках вам даны пары чисел x, y ($-10^9 \le x, y \le 10^9$) — координаты точек.

Формат вывода

Для каждой точки выведите количество отрезков над ней.

1 Описание

Требуется для набора горизонтальных отрезков и набора точек определить, сколько отрезков лежит строго над каждой точкой. Отрезок задан тройкой (l, r, h) и содержит все точки (x, h) при $l \le x \le r$. Точка (x, y) лежит под отрезком, если $l \le x \le r$ и $h > y$. Ограничения: $1 \le n, m \le 10^5$, координаты по модулю не превосходят 10^9. Запросы должны обрабатываться онлайн, то есть по одному, после построения структуры данных по всем отрезкам.

Задача сводится к двум условиям: отрезок должен накрывать абсциссу x и его высота должна быть больше y. Если обрабатывать запросы в произвольном порядке, нужно отвечать на вопрос «сколько отрезков, накрывающих x, имеют высоту больше y», причём для любого x и в любом порядке. Для этого используется **персистентное дерево отрезков**.

Идея решения

Применим метод заметающей прямой по оси x. Каждый отрезок $[l, r]$ порождает два события:

- в точке l отрезок «открывается»: в дерево добавляется $+1$ в позицию, соответствующую высоте h;
- в точке $r + 1$ отрезок «закрывается»: в ту же позицию добавляется -1.

Закрытие в $r + 1$ (а не в r) превращает отрезок в полуоткрытый $[l, r + 1)$. Благодаря этому отрезок учитывается и при $x = l$, и при $x = r$, а порядок событий с одинаковой координатой x не имеет значения: для запроса используются все события с координатой $\le x$.

Высоты предварительно сжимаются: после сортировки и удаления дубликатов каждой высоте соответствует номер от 0 до $k - 1$, где $k \le n$ — число различных высот. Дерево отрезков строится над индексами высот и хранит в каждой вершине количество «открытых» отрезков с высотами из соответствующего диапазона.

События сортируются по x. После применения каждого события создаётся новая версия дерева, поэтому версия номер v описывает состояние после первых v событий; версия 0 — начальное дерево. При изменении копируются только вершины на пути от корня до листа, остальные поддеревья используются совместно со старой версией. Так версия создаётся за $O(\log n)$ времени и памяти, а все предыдущие остаются доступными.

Обработка запроса

Для запроса (x, y):

1. бинарным поиском (`upper_bound`) находится число событий v с координатой $\le x$; нужная версия — `roots[v]`;
2. бинарным поиском по сжатым высотам находится номер p первой высоты, строго большей y; если такой высоты нет, ответ равен 0;
3. в версии `roots[v]` вычисляется сумма на суффиксе индексов $[p, k-1]$: при спуске к листу p каждый раз, когда идём влево, прибавляется значение правого сына.

Сложность

Пусть n — число отрезков, m — число запросов.

- Сортировка событий и высот: $O(n \log n)$.
- Построение всех версий: $2n$ событий по $O(\log n)$ вершин на каждое, то есть $O(n \log n)$ времени и памяти.
- Один запрос: два бинарных поиска и один спуск по дереву, то есть $O(\log n)$. Итого на все запросы $O(m \log n)$.

Память: примерно $2n \cdot (\lceil \log_2 n \rceil + 1) \approx 3{,}6 \cdot 10^6$ вершин по 12 байт, то есть около 43 МБ.

2 Исходный код

Программа состоит из следующих частей.

Вершина дерева отрезков хранится в структуре `Node`: индексы левого и правого сына и количество отрезков в поддереве. Все вершины лежат в одном массиве `pool` (`std::vector<Node>`), а ссылки между вершинами представлены индексами типа `int`, а не указателями. Это уменьшает размер вершины с 24 до 12 байт и снимает вопрос ручного освобождения памяти: она освобождается автоматически при завершении программы.

Функция `createNode` строит начальное (нулевое) дерево. Функция `add` создаёт новую версию: копирует вершины на пути к листу, а второго сына на каждом шаге берёт из старой версии без копирования. Результат рекурсивного вызова сначала сохраняется в локальную переменную и только потом записывается в `pool`, потому что внутри вызова вектор может перевыделить память, и ранее полученная ссылка на элемент стала бы недействительной. Функция `count_height` считает сумму на суффиксе индексов высот.

Структура `Event` хранит координату x события, высоту отрезка и изменение ± 1. В функции `main` читаются отрезки, формируются и сортируются события, сжимаются высоты, строятся версии дерева и затем по одному обрабатываются запросы.

```
#include <algorithm>
#include <iostream>
#include <iterator>
#include <vector>

struct Node {
    int left = -1;
    int right = -1;
    int count = 0;
};

std::vector<Node> pool;

int createNode(int l, int r) {
    int id = pool.size();
    pool.push_back(Node());
    if (l == r) {
        return id;
    }
    int mid = (l + r) / 2;
    int left = createNode(l, mid);
    int right = createNode(mid + 1, r);
    pool[id].left = left;
    pool[id].right = right;
    return id;
```

```
}

int add(int old_node_id, int l, int r, int pos, int delta) {
    int new_node_id = pool.size();
    pool.push_back(Node());
    pool[new_node_id].count = pool[old_node_id].count + delta;
    if (l == r) {
        return new_node_id;
    }
    int mid = (l + r) / 2;
    int left, right;
    if (pos <= mid) {
        left = add(pool[old_node_id].left, l, mid, pos, delta);
        right = pool[old_node_id].right;
    } else {
        left = pool[old_node_id].left;
        right = add(pool[old_node_id].right, mid + 1, r, pos, delta);
    }
    pool[new_node_id].left = left;
    pool[new_node_id].right = right;
    return new_node_id;
}

int count_height(int node_id, int l, int r, int pos) {
    int c = 0;
    int mid = (l + r) / 2;
    while (l < pos) {
        if (pos <= mid) {
            r = mid;
            c += pool[pool[node_id].right].count;
            node_id = pool[node_id].left;
        } else {
            l = mid + 1;
            node_id = pool[node_id].right;
        }
        mid = (l + r) / 2;
    }
    return c + pool[node_id].count;
}

struct Event {
    int x, h, delta;
};

int main(void) {
    int n, m;

    std::cin >> n >> m;

```

```
75      std::vector<Event> events;
76      std::vector<int> heights;
77      events.reserve(2 * n);
78      for (int i = 0; i < n; ++i) {
79          int l, r, h;
80          std::cin >> l >> r >> h;
81          events.push_back({l, h, 1});
82          events.push_back({r+1, h, -1});
83          heights.push_back(h);
84      }
85
86      std::sort(
87          events.begin(),
88          events.end(),
89          [](const auto& a, const auto& b) {return a.x < b.x;}
90      );
91
92      std::sort(heights.begin(), heights.end());
93      heights.erase(
94          std::unique(heights.begin(), heights.end()),
95          heights.end()
96      );
97
98      std::vector<int> roots;
99      roots.reserve(2 * n + 1);
100     roots.push_back(createNode(0, n-1));
101     for (auto& event : events) {
102         auto it = std::lower_bound(heights.begin(), heights.end(), event.h);
103         roots.push_back(add(
104             roots.back(),
105             0,
106             n-1,
107             std::distance(heights.begin(), it),
108             event.delta
109         ));
110     }
111
112     for (int i = 0; i < m; ++i) {
113         int x, y;
114         std::cin >> x >> y;
115
116         auto it_event = std::upper_bound(
117             events.begin(),
118             events.end(),
119             x,
120             [](const auto& val, const auto& b){return val < b.x;}
121         );
122         auto v = std::distance(events.begin(), it_event);
123
```

```
124         auto it_height = std::upper_bound(heights.begin(), heights.end(), y);
125         if (it_height == heights.end()) {
126             std::cout << 0 << '\n';
127             continue;
128         }
129         auto pos = std::distance(heights.begin(), it_height);
130 
131         std::cout << count_height(roots[v], 0, n-1, pos) << '\n';
132     }
133 }
```

В таблице приведено краткое описание основных элементов программы.

main1.cpp	
struct Node	Вершина дерева отрезков: индексы сыновей и счётчик
struct Event	Событие заметающей прямой: координата, высота, изменение ± 1
int createNode(int l, int r)	Строит пустое дерево на отрезке индексов $[l, r]$, возвращает индекс корня
int add(int old_node_id, int l, int r, int pos, int delta)	Создаёт новую версию дерева с прибавлением `delta` в позиции `pos`, возвращает индекс нового корня
int count_height(int node_id, int l, int r, int pos)	Возвращает сумму значений на индексах $[pos, r]$ в данной версии
int main()	Чтение данных, построение версий, обработка запросов

3 Консоль

Запуск на примере из условия задачи:

```
$ g++ -pedantic -Wall -Wextra -std=c++17 -O2 -o sol main1.cpp
$ cat test1
5 3
0 5 2
1 3 1
1 8 5
-2 0 3
-1 6 -1
1 0
3 -3
3 5
$ ./sol <test1
3
4
0
```

Разберём первый запрос $(1, 0)$: точку $x = 1$ накрывают отрезки с высотами 2, 1, 5 и -1 (отрезок $[-2, 0]$ высоты 3 её не накрывает). Строго выше $y = 0$ лежат отрезки с высотами 2, 1 и 5, поэтому ответ равен 3. Для запроса $(3, -3)$ точку накрывают отрезки высот 2, 1, 5 и -1, и все они выше -3, ответ 4. Для запроса $(3, 5)$ нет отрезка выше высоты 5, ответ 0.

4 Тест производительности

Для проверки производительности решение сравнивается с наивным алгоритмом, который для каждого запроса перебирает все отрезки и проверяет условие $l \le x \le r$ и $h > y$. Его сложность равна $O(nm)$, а у реализованного решения $O((n+m)\log n)$. Цель теста — убедиться, что теоретические оценки подтверждаются на практике: при увеличении размера входных данных время работы перебора должно расти квадратично, а время работы персистентного дерева отрезков — почти линейно.

Условия эксперимента

Размеры входных данных брались равными $n = m$ и удваивались от 1000 до 64000. Левые и правые границы отрезков, высоты и координаты точек выбирались случайно из диапазона $[-10^9, 10^9]$ при помощи генератора со значением `seed` $= 1$, поэтому почти все высоты различны, и дерево не вырождается. Обе программы компилировались командой `g++ -O2 -std=c++17`. Для каждого размера решение запускалось 3 раза, а наивный алгоритм — 3 раза при $n < 50000$ и один раз при $n \ge 50000$; в таблице приведена медиана времени работы процесса. Время включает чтение входных данных, вывод результата направлялся в `/dev/null`.

Проверка корректности

Перед замерами ответы решения сравнивались с ответами наивного алгоритма на 200 случайных малых тестах ($n, m \le 8$, координаты по модулю не более 6, поэтому часто встречаются совпадающие координаты и высоты). Расхождений нет.

```
$ g++ -O2 -std=c++17 -o sol main1.cpp
$ g++ -O2 -std=c++17 -o brute brute.cpp
$ ./check.sh
OK: 200 малых тестов совпали
```

Результаты

```
$ python3 bench.py 1000 2000 4000 8000 16000 32000 64000
n=m     sol,c   brute,c
1000     0.0082     0.0093
2000     0.0124     0.0202
4000     0.0214     0.0595
8000     0.0405     0.2062
16000     0.0782     0.7886
```

```
32000      0.1953      2.9173
64000      0.3651     11.0841
```

В таблице ниже для каждого удвоения n указано, во сколько раз выросло время по сравнению с предыдущей строкой. При линейной зависимости ожидается значение около 2, при квадратичной — около 4.

$n = m$	Решение, с	Рост	Перебор, с	Рост
1000	0,0082	—	0,0093	—
2000	0,0124	1,51	0,0202	2,17
4000	0,0214	1,73	0,0595	2,95
8000	0,0405	1,89	0,2062	3,47
16000	0,0782	1,93	0,7886	3,82
32000	0,1953	2,50	2,9173	3,70
64000	0,3651	1,87	11,0841	3,80

Выводы по результатам

- **Наивный перебор** при каждом удвоении n замедляется в 3,5–3,8 раза (при $n \geq 8000$), то есть примерно в 4 раза, как и должно быть при сложности $O(n^2)$ (при $n = m$).
- **Персистентное дерево отрезков** при каждом удвоении n замедляется примерно в 1,9 раза (кроме выброса 2,5 при $n = 32000$, который, вероятно, объясняется шумом измерения), то есть примерно в 2 раза с небольшой добавкой от множителя $\log n$. Это согласуется с оценкой $O((n + m) \log n)$.
- При малых n (порядка тысяч) времена обоих алгоритмов сопоставимы, а рост времени отличается от теоретического: в этих замерах заметную долю составляют запуск процесса и ввод-вывод, а не сами вычисления. По мере роста n эти накладные расходы перестают влиять, и картина становится близкой к теоретической.
- При $n = m = 64000$ решение работает за 0,37 с, а наивный перебор — за 11,1 с, то есть примерно в 30 раз медленнее. Так как рост времени перебора квадратичный, а решения — почти линейный, разрыв увеличивается с ростом n (при $n = 8000$ разница около 5 раз, при $n = 16000$ — около 10 раз).

Таким образом, эксперимент подтвердил оценки сложности: $O(nm)$ для наивного алгоритма и $O((n + m) \log n)$ для персистентного дерева отрезков.

5 Выводы

В ходе работы я познакомился с персистентными структурами данных и реализовал персистентное дерево отрезков. Главное, что я понял: чтобы сохранить доступ ко всем прошлым версиям структуры, не нужно копировать её целиком. При изменении достаточно скопировать путь от корня до изменяемого листа ($O(\log n)$ вершин), а остальные поддеревья использовать совместно со старой версией. Этот приём называется копированием пути.

Я научился сводить геометрическую задачу (сколько отрезков лежит над точкой) к запросам на версиях дерева с помощью метода заметающей прямой: события открытия и закрытия отрезков превращаются в версии, а запрос по координате x — в выбор нужной версии бинарным поиском. Заодно я разобрал типичную ошибку с граничными случаями: если закрывать отрезок в точке r, то при равных координатах событий результат зависит от порядка сортировки, а закрытие в точке $r + 1$ (полуоткрытый отрезок) делает решение независимым от этого порядка.

Отдельно я разобрался с управлением памятью в персистентных структурах. Вершины общие у нескольких версий, поэтому рекурсивное удаление от каждого корня приводит к повторному освобождению, а `shared_ptr` слишком расточителен по памяти и времени. Для задачи, где все версии живут до конца программы, лучше всего подошла арена: все вершины лежат в одном `std::vector`, а ссылки между ними являются индексами. Это сократило расход памяти (вершина занимает 12 байт вместо 24) и избавило от ручного освобождения. При этом пришлось учесть, что при перевыделении вектора ссылки на его элементы становятся недействительными.

Практическое сравнение показало разницу между асимптотиками $O(nm)$ и $O((n + m) \log n)$: на максимальных данных решение работает за доли секунды, а наивный перебор — почти минуту. Подобные структуры пригодятся в задачах с запросами к прошлым состояниям данных (запросы вида «на момент времени t»), в онлайн-вариантах задач на подсчёт в диапазонах, а также в системах контроля версий и баз данных, где неизменяемые структуры с общими частями применяются очень широко.

Список литературы

[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))

[2] *Персистентные структуры данных — Викиконспекты.* URL: `https://neerc.ifmo.ru/wiki/index.php?title=Персистентные_структуры_данных` (дата обращения: 04.11.2026).